

## **Abstract:**

### **Auctioneer**

This project focuses on developing a Real-Time Auction Platform that enables seamless interaction between sellers, users (bidders), and an admin panel. The platform offers a dynamic environment where sellers can list items for auction, users can participate in real-time bidding, and the admin can oversee and manage the auction process. The core functionality involves handling real-time bidding events with high accuracy and speed, ensuring that users experience a live, competitive bidding atmosphere.

The platform is built using the MERN stack:

1. **Frontend:** Developed using React, offering an intuitive and responsive user interface for bidders, sellers, and the admin.
2. **Backend:** Powered by Node.js and Express, ensuring robust API services, real-time event handling, and smooth data flow.
3. **Database:** MongoDB is used to store auction data, user profiles, bids, and auction history in a scalable and efficient manner.

Key features of the platform include:

**Real-Time Bidding:** Leveraging WebSockets (Socket.io) to broadcast real-time auction updates to all participants.

### **User Roles:**

**Sellers** can create, update, and manage auction listings.

**Users** can place bids in real time and track ongoing auctions.

**Admin** can manage users, sellers, and auction settings, ensuring fair operation.

### **Users**

#### **1.Sellers**

- List items for auction, set starting prices, and track bidding activity on their listings.
- Create and manage auction listings.

- Add descriptions, images, and bid parameters (e.g., starting price, reserve price).
- View auction performance and finalize the sale after the auction ends.

## **2. Bidders (Users)**

**Participate in auctions by placing real-time bids on items of interest.**

- Browse and search for auction items.
- Place live bids during ongoing auctions.
- Receive real-time notifications (e.g., outbid alerts, auction results).
- Make payments after winning an auction.

## **3. Admin**

**Manage and oversee the auction platform to ensure smooth operations and fairness.**

- Approve or reject auction listings.
- Monitor real-time auctions and resolve disputes.
- Handle platform configurations, user account management, and analytics reporting.

## **Modules**

### **1. Seller Module**

- Functionality for sellers to create, edit, and manage their auction listings.
- Tools to upload item details, set bidding parameters, and view auction performance.

### **2. Bidder Module**

- Interface for bidders to browse items, participate in live auctions, and receive notifications.
- Real-time updates for bids and auction outcomes.

### **3. Admin Module**

- Dashboard for administrators to oversee the platform, manage users, monitor auctions, and resolve disputes.
- Analytics and reporting tools for tracking platform performance.

### **4. Real-Time Bidding Engine**

- Core system handling live bidding events with high accuracy and minimal latency.
- Includes conflict resolution to manage simultaneous bids.

### **5. Notification System**

- Sends real-time alerts to users (e.g., outbid notifications, auction reminders, winner announcements).

### **6. Payment Integration Module**

- Securely handles transactions for winning bidders and processes seller payouts.

### **7. Search and Filtering Module**

- Enables users to search and filter auction items by category, price, and other criteria.

### **8. User Authentication and Profile Module**

- Provides secure login, registration, and account management for sellers, bidders, and admins.

## **. INTRODUCTION**

Planning and scheduling are distinct but inseparable aspects of managing a successful project.

- The process of planning primarily deals with selecting the appropriate policies and procedures to achieve the objectives of the project.
- Scheduling converts the project action plans for scope, time, cost, and quality into an operating timetable.
- Translating the project criteria for scope, time, cost, and quality, along with the requirements for human resources, communications, risk, and procurement, into a workable "machinery" makes scheduling a critical interface for the project team.
- Taken together with the project plan and budget, the schedule becomes the major tool for managing projects.
- Additionally, the integrated cost-time schedule serves as the fundamental basis for monitoring and controlling project activity throughout its life cycle.

This paper addresses the integrated processes of planning and scheduling of multidisciplinary programs. It presents a summary of major Project Management topics and details a systematic process for transforming the Project Plan into the Project Schedule for project control. Intended for project management novices, the paper concludes with a suggested professional development scheme.

## **9.2. PLANNING A SOFTWARE PROJECT**

Project planning is at the heart of the project life cycle, defining where you're going and how you're going to get there.

- The planning phase is when the project plans are documented, deliverables and requirements are defined, and the schedule is created.
- It involves creating a set of plans to help guide your team through the implementation and closure phases.
- Plans created during this phase assist in managing time, cost, quality, changes, risks, and related issues.

- These plans also help control staff and external suppliers, ensuring that the project is delivered on time, within budget, and on schedule.

The purpose of the project planning phase is to:

- Establish business requirements.
- Establish cost, schedule, list of deliverables, and delivery dates.
- Establish resource plans.
- Obtain management approval and proceed to the next phase.

The basic processes of project planning are:

- **Scope planning** – Specifying the in-scope requirements for the project to facilitate creating the work breakdown structure.
- **Preparation of the work breakdown structure** – Spelling out the breakdown of the project into tasks and sub-tasks.
- **Project schedule development** – Listing the entire schedule of the activities and detailing their sequence of implementation.
- **Resource planning** – Indicating who will do what work, at which time, and if any special skills are needed to accomplish the project tasks.
- **Budget planning** – Specifying the budgeted cost to be incurred at the completion of the project.
- **Procurement planning** – Focusing on vendors outside your company and subcontracting.
- **Risk management** – Planning for possible risks and considering optional contingency plans and mitigation strategies.
- **Quality planning** – Assessing quality criteria to be used for the project.

### 9.2.1. STEPS INVOLVED IN PLANNING A SYSTEM

SDLC (Software Development Life Cycle) is a process followed for a software project within a software organization. It consists of a detailed plan describing how to develop, maintain, replace, and alter or enhance specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.

A typical **Software Development Life Cycle (SDLC)** consists of the following stages:

- 1. **Planning and Requirement Analysis**
- 2. **Defining Requirements**
- 3. **Designing the Product Architecture**
- 4. **Building or Developing the Product**
- 5. **Testing the Product**
- 6. **Deployment in the Market and Maintenance**

**PROJECT SCHEDULING**

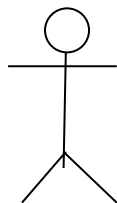
| TASK                 | START DATE | END DATE   |
|----------------------|------------|------------|
| SYSTEM STUDY         | 10-01-2025 | 13-01-2025 |
| INITIAL REPORT       | 14-01-2025 | 15-01-2025 |
| TOPIC SUBMISSION     | 16-01-2025 | 17-01-2025 |
| SYSTEM SELECTION     | 18-01-2025 | 20-01-2025 |
| SELECTING TECHNOLOGY | 21-01-2025 | 22-01-2025 |
| TABLE DESIGN         | 23-01-2025 | 30-01-2025 |
| DATA MODELS          | 31-01-2025 | 15-02-2025 |
| FORM DESIGN          | 17-02-2025 | 22-02-2025 |
| CODING               | 24-02-2025 | 22-03-2025 |
| TEST CASE DESIGN     | 24-03-2025 | 11-04-2025 |
| SYSTEM TESTING       | 15-04-2025 | 28-04-2025 |
| UPDATES              | 29-04-2025 | 09-05-2025 |

## USECASE DIAGRAM

The most important aspect is to capture the dynamic behaviour when modelling a system. Dynamic means the behaviour of the system when it is operating. Use case diagram is one of the diagrams that describe the dynamic behaviour of the system. Use case diagrams are used to gather the requirements of a system including internal and external influences. These requirements are mostly design requirements. Hence, when a system analysed to gather its functionalities, use cases are prepared and actors are identified. When the initial task is complete, use case diagrams are modelled to present the outside view.

### Symbols in Use-case Diagram:

An actor represents a coherent set of roles that users of a system play when interacting with the use cases of the system. Actors can be anything-humans, devices, other systems. One physical object can play several roles and so can be modelled by several actors.



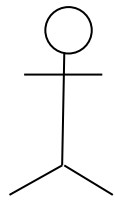
Use cases describe what a system does, not how it does it. A use case contains multiple scenarios, each of which describes a specific flow of events through the use case. Use case behavior is specified by describing the scenarios clearly enough for outsiders to understand.



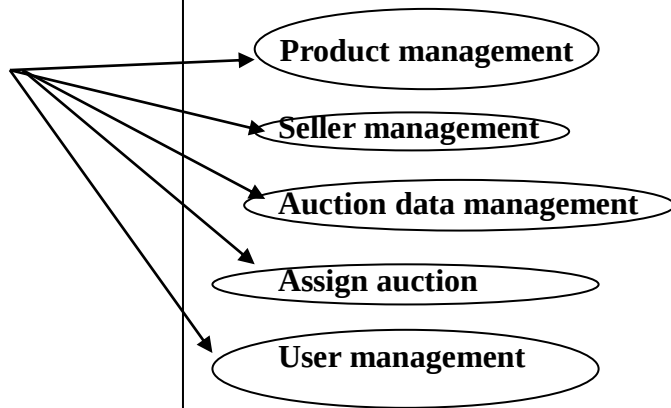
Connection symbol represent the connection between actor and use cases.



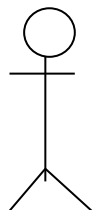
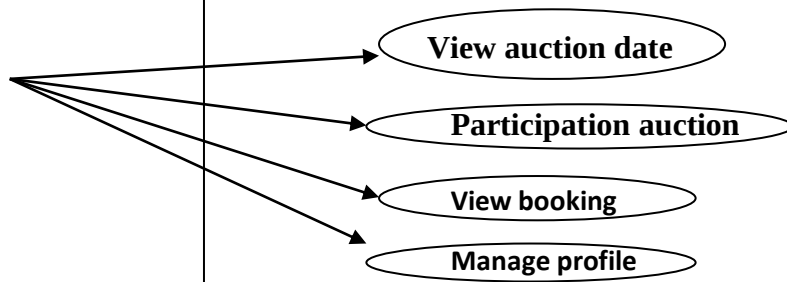
Include symbol include the relationship between use cases. One use case(base) includes the functionality of another (inclusion case).



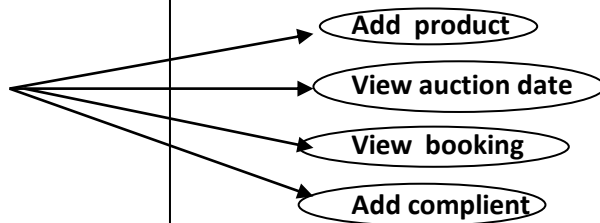
**Admin**



**User**



**Seller**



# ACTIVITYDIAGRAM

Activity diagrams are graphical representation of workflow of stepwise activities and actions with support for choice, iteration and concurrency. UML, activity diagrams are intended to model both computational and organizational process. It shows the overall flow of control.

Symbols in Activity Diagram:-

## Action State:-

It is an automatic state once it started execution. It will come to completion without any interruption at the middle of execution.

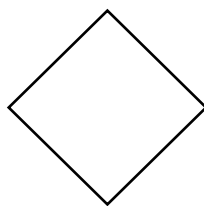


## Transition:-

When the action or activity state completes the flow of control passes immediately to the next action or activity state. This is represented by using the solid lines.

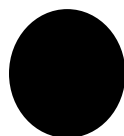


**Branching:-** It specifies the alternative paths that are taken based on some Boolean expressions.



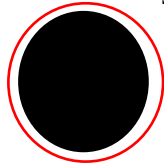
## Initial node:-

The initial node is a control node from where the activity is invoked. An activity may have more than one initial node.

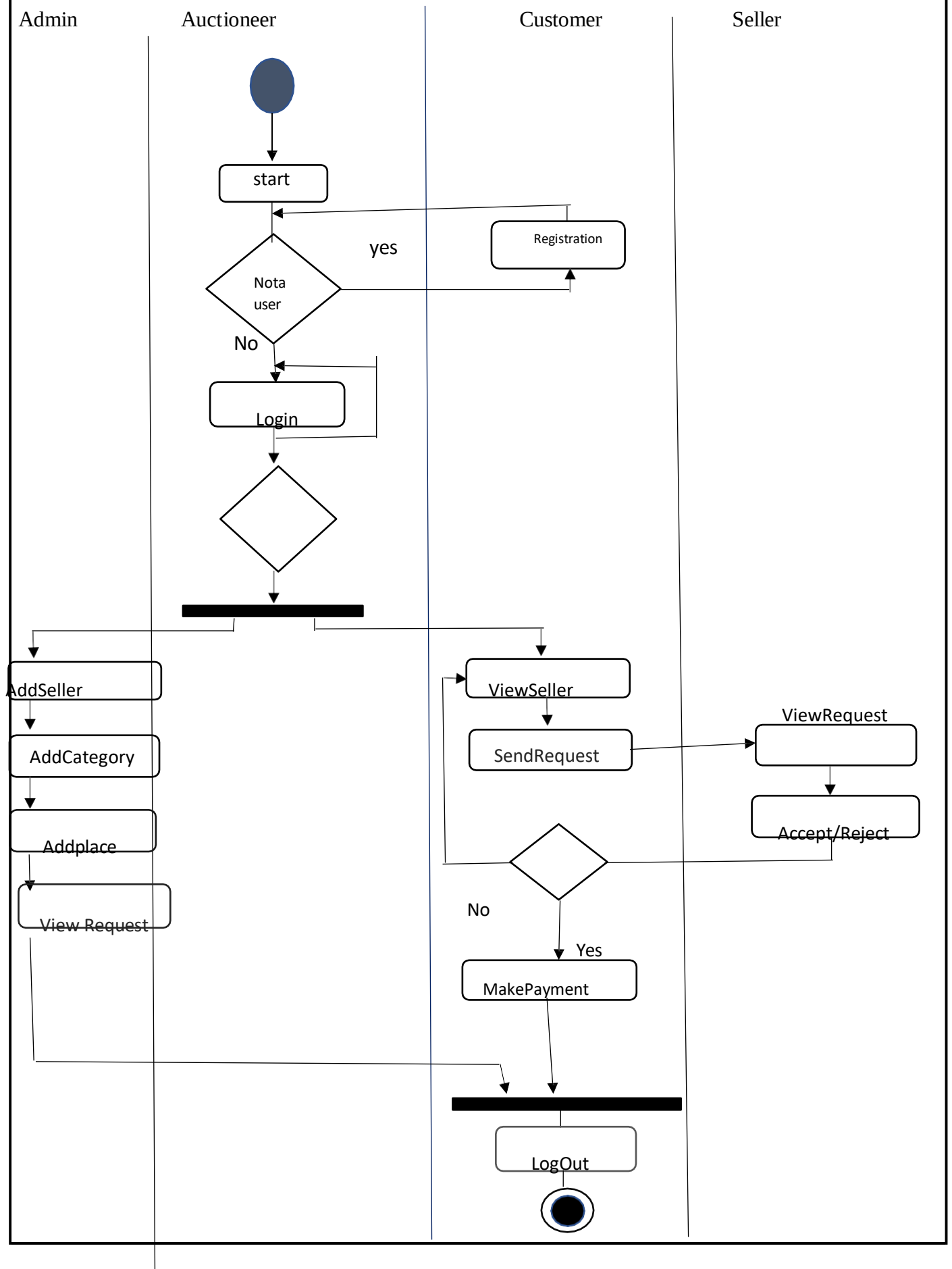


**Final node:-**

The final nodes show the end of the activity. An activity can have more than one final node but the first one reached will stop the activity.



# ACTIVITYDIAGRAM

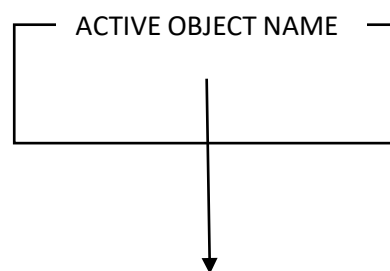


## 2 SEQUENCE DIAGRAM

A sequence diagram is an interaction diagram that shows how objects operate with one another and in what order. It is a constant of a message sequence chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the logical view of the system under development.

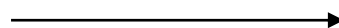
### Symbols in Sequence Diagram:-

- **Life lines & life line boxes:** - Active objects can be an object which is an instance of a class and can be drawn with rectangular box called the life line box with its name is specified within the box. The long dashed line tailing the object is called life lines.



- **Message:** -It is used to illustrate the communication between different active objects. Each message between object is represented with a message, expressions and filled arrow solid lines from the calling active objects life line to the recipient life lines.

- **Synchronous:** - This type of message is used when it is important that a message is received & completed before execution of control flow begins.



- **Asynchronous:** - This type of message is sent from an active object and waits for a responds. This type of message is used when control flow doesn't lead to interrupted before complete the operation.



- Flat:** -This type of message is used when there is no distinction between asynchronous and synchronous messages

## SEQUENCEDIAGRAM

